Sharding does. To solve this problem, you can easily create a sharded entity from an `EventSourcedBehavior` just as you would with a simple actor: by adding an `EntityTypeKey`.

Another scenario is two different types of entities with the same ID. Again, Akka Sharding is used to solve this problem. In this case, the type key of the entity is appended to the ID of the persistent actor. This combination guarantees the uniqueness of the entity.

The `state` is the simplest of the four parameters. Its type (`State`) must be defined as a class when writing the persistent actor to provide a placeholder for the current state. In the container, the `State` is a list of cargo:

```
final case class State(cargos: List[Cargo] = Nil)
```

The function `commandHandler (State, Command) => Effect[Event, State]` takes as input any message of type `Command` sent to the actor, as well as its current `State`. From this, it creates an `Event` and a new `State`. Commands create reactions on the actor—called *effects*—such as persist, reply, and stop. Depending on the received message and the actor's state, you can pattern-match on both to achieve the desired effects.

Side effects—which are different from effects—are, for example, a post to an HTTP server or a database insert and should also be executed in the command handler. With the `thenRun` function, the side effect can be added to effects like `persist`:

```
Effect.persist([Event]).thenRun(//**side effect**//).   ❶
```

❶ Side effect such as a database insertion

When the command handler has successfully persisted the event, the `eventHandler` function is executed (you don't want to change the actor's state if the command handler has not validated the event). This has to do with the fact that saved events will be processed again by the event handler when the persistent actor is restarted on recovery. Only what is saved can determine the current state of the actor.

When the event is persisted, the `eventHandler` function receives the event and the state from the command handler, which can now update the actor's state. The `eventHandler` function has the following signature:

```
def eventHandler(state: State, event: Event): State
```

Be careful to avoid side effects in the event handler. When a persistent actor is restarted, each entry is replayed to return to its last state by being passed as input to the `eventHandler` function. Thus, if the event handler has side effects, they will also be replayed.